

Adapting *mcp-remote* for SSE in Claude Desktop Integration

1. Transport Flag for True SSE

To force a genuine Server-Sent Events transport, use the `--transport sse-only` option. This ensures *mcp-remote* uses SSE exclusively (no HTTP fallback) ¹. For example:

```
"args": ["mcp-remote", "<MCP_URL>", "--transport", "sse-only", "--header",  
"x-api-key: ..."]
```

(Note: You could also use `sse-first` to prefer SSE with HTTP as fallback, but `sse-only` guarantees a pure SSE connection ².)

2. SSE Support in *mcp-remote*

Yes – *mcp-remote* natively supports SSE as a transport. It offers multiple transport strategies including SSE. By default it uses `http-first` (tries HTTP, then SSE on 404) ³, but you can explicitly choose SSE via flags (as above). The tool is designed to handle SSE streaming out of the box; Anthropic's docs even provide an example of using `--transport sse` for connectors ⁴.

3. Accept Header Requirements

You do **not** need to manually add an `Accept: text/event-stream` header – *mcp-remote* sets this automatically when using SSE. According to the MCP spec, a client must include an `Accept` header advertising SSE for both POST and GET requests ⁵ ⁶. The *mcp-remote* client adheres to this spec, so it will send the proper `Accept: application/json, text/event-stream` for POSTs and `Accept: text/event-stream` for SSE listen connections. In practice, when you specify the SSE transport, the CLI handles those headers internally (no extra `--header` flag needed for SSE negotiation).

4. Endpoint URL: `/mcp` vs `/sse`

This depends on how the MCP server is set up: many hosted servers expose a specific SSE endpoint (often suffixed with `/sse`). For example, Anthropic's integration docs use `.../sse` for Asana's SSE endpoint ⁷. If your server provides a dedicated SSE URL, use that. Otherwise, if the server uses a single endpoint for both transports, the same path (e.g. `/mcp`) works with SSE as long as the client requests SSE. In the Netdata example, the *mcp-remote* was pointed at the same `/mcp` URL with an `--sse` flag and it functioned as an SSE stream ⁸. **In short:** use the `/sse` endpoint if one exists; if not, use the regular endpoint and rely on the SSE transport flag to initiate a streaming connection.

5. Claude Desktop SSE Integration Quirks

Claude Desktop (free tier) only supports launching local MCP processes over stdio – it **cannot directly speak HTTP/SSE** without a bridge ⁹. The workaround (which you’re using) is to run `mcp-remote` as a local proxy so Claude thinks it’s a local stdio server while `mcp-remote` handles the remote SSE connection ¹⁰. A few specific quirks to note:

- **Custom Headers:** The official *Custom Connectors* UI in Claude (for paid plans) doesn’t allow arbitrary headers – it expects OAuth or a token input ¹¹. In the JSON config method, however, using `mcp-remote --header` is the way to send API keys or Bearer tokens (as you’ve done). This is a common approach to pass an API key for SSE endpoints that require one ¹².
- **Endpoint and Transport:** Ensure you match the server’s expectations. If the server’s discovery docs or examples show a `/sse` endpoint, use it in the config. And pair it with `--transport sse-only` (or the shorthand `--sse`) to make Claude Desktop use a standards-compliant SSE streaming connection. Anthropic’s own docs note that SSE transport still works but is considered legacy in favor of “streamable HTTP” ⁴, so Claude Desktop will happily use SSE mode when configured – just be sure to use the latest `mcp-remote` version for best compatibility and security (v0.1.16+ is recommended due to prior SSE bugs) ¹³.
- **Restart & Environment:** After editing the config, restart Claude Desktop for changes to take effect. Also verify your system has Node 18+ (Claude Desktop uses the system Node for `mcp-remote`) ¹⁴. If your SSE server is on HTTP (no TLS), include `--allow-http` in the args to let `mcp-remote` connect to it ¹⁵.

Bottom line: Update your `claude_desktop_config.json` entry to use `--transport sse-only` (or `--sse`), point it at the SSE-enabled URL (adding `/sse` if required by that server), and keep the API key header. For example:

```
{
  "mcpServers": {
    "leowiki": {
      "command": "npx",
      "args": [
        "mcp-remote@latest",
        "--sse",
        "https://leowiki-mcp.stream/sse",          // use /sse if available
        "--header", "x-api-key: ${LEOWIKI_API_KEY}"
      ]
    }
  }
}
```

This will make Claude Desktop establish a streaming SSE connection to the MCP server (on the same `/mcp` endpoint or the `/sse` endpoint as needed), without using the `http-first` fallback path. The `mcp-remote` proxy will automatically include the proper SSE headers and handle the stream. Adjusting the config in this way should achieve a standards-compliant SSE integration with your Claude Desktop instance.

Sources: The MCP Remote README and Anthropic docs confirm the above details about transport flags and SSE usage [3](#) [5](#) [7](#) [8](#) [9](#) .

[1](#) [2](#) [3](#) [14](#) GitHub - geelen/mcp-remote

<https://github.com/geelen/mcp-remote>

[4](#) Connect Claude Code to tools via MCP - Claude Code Docs

<https://code.claude.com/docs/en/mcp>

[5](#) [6](#) Transports - Model Context Protocol

<https://modelcontextprotocol.io/specification/2025-06-18/basic/transports>

[7](#) Enable MCP in Claude Desktop: Step-by-Step Guide | Natoma

<https://natoma.ai/blog/how-to-enabling-mcp-in-claude-desktop>

[8](#) [9](#) [11](#) [15](#) Claude Desktop | Learn Netdata

<https://learn.netdata.cloud/docs/netdata-ai/mcp/mcp-clients/claude-desktop>

[10](#) [12](#) [13](#) Bridging MCP Transports: Turning an STDIO Server into SSE (and the Other Way Around) | by Sofiane Ghadab | Medium

<https://thesof.medium.com/bridging-mcp-transports-turning-an-stdio-server-into-sse-and-the-other-way-around-da23a36e074a>